

I/O Devices and Device Controllers

1 I/O Devices

Basic Idea

I/O devices connect a computer to input, output, storage, communication, or user interaction.

- Examples: disks, keyboards, mice, displays, network cards, printers, scanners, cameras, and buses.
- Devices differ by speed, data format, access method, and control mechanism.
- The OS hides many hardware details behind general device models.

2 Block Devices

Definition

A **block device** stores data in fixed-size addressable blocks. Transfers happen one or more complete blocks at a time.

Property	Meaning
Block size	Usually 512 bytes to 65,536 bytes.
Block address	Each block has a number or address.
Independent access	Any block can be read or written separately.
Examples	Hard disks, SSDs, USB sticks, and Blu-ray discs.

Key Point

Block devices support direct access to a named block.

3 Character Devices

Definition

A **character device** sends or receives a byte stream without addressable blocks.

Property	Meaning
Stream-oriented	Data is processed in sequence.
No block address	The OS cannot choose block N .
No seek operation	The OS cannot jump to an arbitrary position.
Examples	Keyboards, mice, printers, network interfaces, and scanners.

Key Point

Character devices are used as streams, not random-access storage.

4 Limits of the Classification

Imperfect Model

The block-device and character-device model is useful, but some devices do not fit cleanly.

Device	Why it is special
Clock	Produces timer interrupts, not blocks or byte streams.
Memory-mapped screen	Acts like memory plus display hardware.
Touch screen	Combines display and input events.
Tape drive	Has blocks, but reaching block N is sequential and slow.

Practical Use

File systems can target abstract block devices while drivers handle hardware-specific details.

5 Device Data Rates

The I/O subsystem must handle both very slow human input and very fast buses or networks.

Device, network, or bus	Typical data rate	Category
Keyboard	10 bytes/sec	Human input
Mouse	100 bytes/sec	Human input
56K modem	7 KB/sec	Communication
Scanner at 300 dpi	1 MB/sec	Input
Digital camcorder	3.5 MB/sec	Media input
4x Blu-ray disc	18 MB/sec	Block storage
802.11n wireless	37.5 MB/sec	Network
USB 2.0	60 MB/sec	Bus
FireWire 800	100 MB/sec	Bus
Gigabit Ethernet	125 MB/sec	Network
SATA 3 disk drive	600 MB/sec	Storage interface
USB 3.0	625 MB/sec	Bus
SCSI Ultra 5 bus	640 MB/sec	Bus
Single-lane PCIe 3.0 bus	985 MB/sec	Bus
Thunderbolt 2 bus	2.5 GB/sec	Bus
SONET OC-768 network	5 GB/sec	Network

Speed Range

I/O rates range from bytes/sec to GB/sec.

6 Device Controllers

Definition

A **device controller**, or **adapter**, is the electronic component that controls an I/O device.

Part	Role
Mechanical component	The physical device.
Electronic component	The controller or adapter.
Location	Motherboard chip or expansion card, such as PCIe.
Scope	One controller may manage multiple similar devices.

Modularity

Standard interfaces let devices and controllers work together without exposing every hardware detail to the OS.

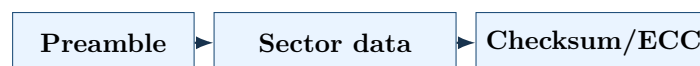
Common interfaces: SATA, SCSI, USB, Thunderbolt, and FireWire.

7 Controller Responsibilities

Main Job

The controller converts raw device signals into blocks, bytes, or events the OS can use.

For a disk, a sector usually has this structure:



Step	Controller action
Receive	Accept the raw bit stream.
Buffer	Assemble bits into bytes or blocks.
Validate	Check checksum or ECC.
Correct	Fix recoverable errors.
Transfer	Copy verified data to main memory.

Key Point

The OS talks to the controller; the controller handles electrical and timing details.

8 Display Controller Example

Without a controller	With a controller
Software controls pixels manually.	The OS sets display parameters and writes display data.
Code depends on device details.	The controller converts memory data into display signals.
Portability is poor.	One software model can support many displays.

Historical Note

LCD screens replaced CRT monitors because they are thinner, lighter, less fragile, and more power efficient.

9 Worked Example

Problem

A disk sector has 512 bytes of payload. How many payload bits must the controller assemble?

Solution

$$512 \text{ bytes} \times 8 \frac{\text{bits}}{\text{byte}} = 4096 \text{ bits.}$$

Answer

The controller assembles 4096 payload bits, excluding preamble, checksum, and ECC bits.

Memory-Mapped I/O

1 Controller Registers

Main Idea

Each device controller has registers that the CPU uses to communicate with the device.

The operating system writes to controller registers to command the device. It reads from them to check device status.

Register use	Meaning
Command	Tell the device to read, write, start, stop, or perform another action.
Status	Check whether the device is ready, busy, finished, or in error.
Data buffer	Hold data being transferred between the device and memory.

Example: a display may use video RAM as a data buffer. Programs or the operating system write pixel data into this buffer, and the display controller uses it to update the screen.

2 Separate I/O Port Space

Definition

In **separate I/O port space**, device registers are addressed by I/O port numbers instead of normal memory addresses.

An I/O port number is commonly an 8-bit or 16-bit integer. The set of all I/O ports is called the **I/O port space**. It is protected so ordinary user programs cannot access it directly.

```
IN  REG,PORT    // read I/O port PORT into CPU register REG
OUT PORT,REG     // write CPU register REG into I/O port PORT
```

In this design, memory addresses and I/O port addresses are separate.

Instruction	Meaning
IN R0,4	Read I/O port 4 into register <i>R0</i> .
MOV R0,4	Read memory word 4 into register <i>R0</i> .

Key Point

The two 4's refer to different address spaces: one is an I/O port address, and the other is a memory address.

3 Memory-Mapped I/O

Definition

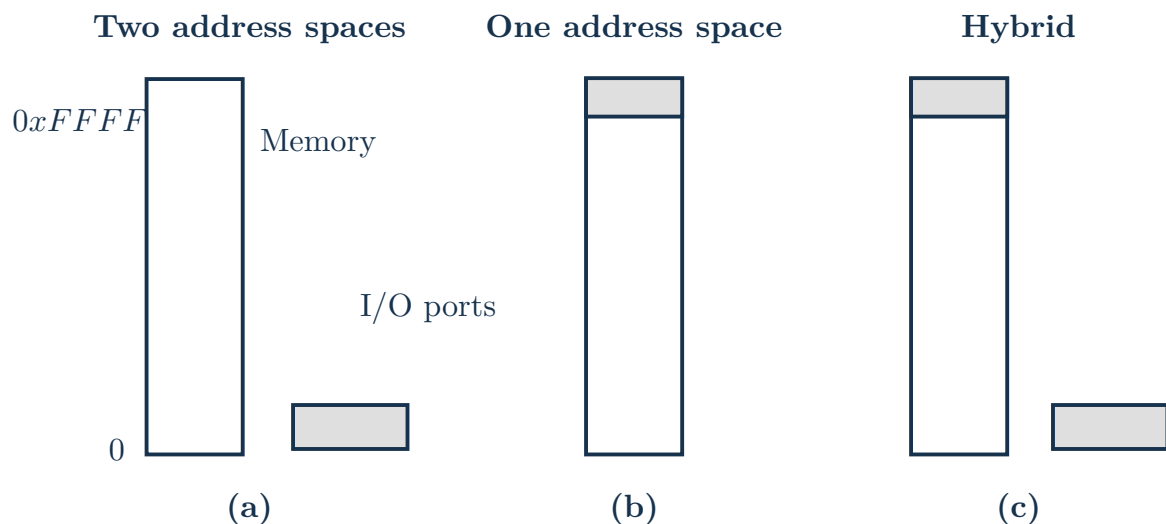
In **memory-mapped I/O**, device control registers are assigned normal memory addresses.

The assigned addresses are not backed by ordinary RAM. Instead, when the CPU reads or writes those addresses, the device controller responds. These addresses are usually placed near the top of the address space.

Simple View

With memory-mapped I/O, device registers look like memory variables to the CPU and to system software.

4 Addressing Schemes



Scheme	Meaning
Separate I/O	Memory and I/O ports use different address spaces.
Memory-mapped I/O	Device registers occupy selected addresses inside the memory address space.
Hybrid	Data buffers are memory-mapped, but control registers use separate I/O ports.

x86 Example

x86 systems use a hybrid approach: some device buffers are memory-mapped, and separate I/O ports are also available.

5 How the CPU Accesses I/O

For a read operation, the CPU places an address on the bus and asserts a read control signal.

Design	Who responds?
Separate spaces	A control signal tells whether the address is for memory or I/O. Memory responds to memory requests; the I/O device responds to I/O requests.
One address space	Memory modules and I/O devices compare the address with their assigned ranges. The matching component responds.

No Address Conflict

In memory-mapped I/O, no address should be assigned to both RAM and an I/O device. Otherwise, two components could respond to the same CPU request.

6 Advantages of Memory-Mapped I/O

Advantage	Explanation
Can use C/C++	Device registers can be accessed like memory variables. Special assembly instructions are not required.
Simpler protection	The OS can keep device-register pages out of user virtual address spaces.
Fine-grained access	The OS can map only selected device pages into a process or driver address space.
Normal instructions work	Any instruction that reads or writes memory can also read or write device registers.

```

LOOP:  TEST PORT4      // check if device register is 0
        BEQ  READY     // if 0, device is ready
        BRANCH LOOP    // otherwise keep testing
READY:

```

Instruction Benefit

With memory-mapped I/O, a register can be tested directly. With separate I/O ports, the value must first be read into the CPU and then tested.

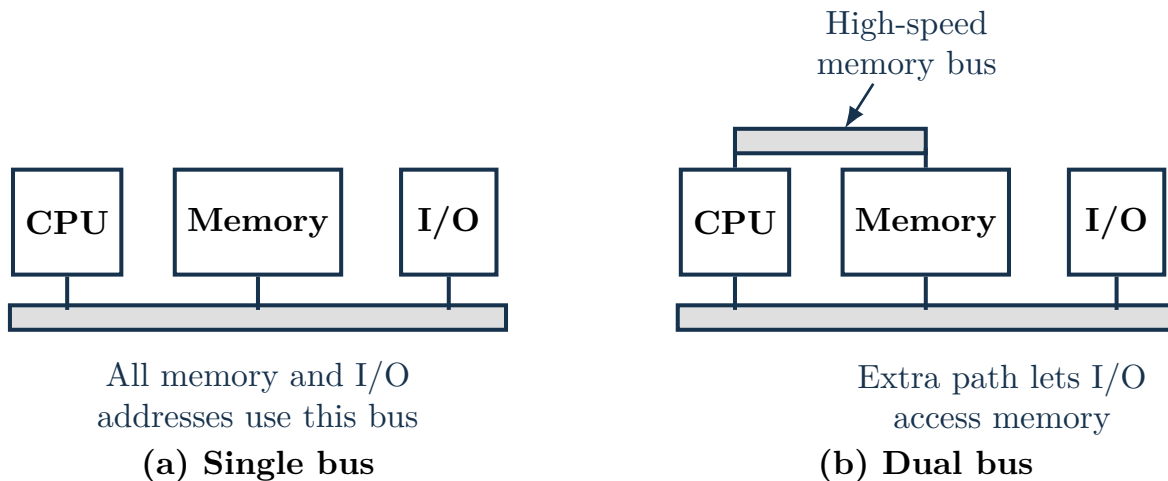
7 Disadvantages of Memory-Mapped I/O

Problem	Explanation
Caching danger	A cached device register may hide a real device-status change.
Hardware complexity	The hardware must disable caching for I/O address ranges, often per page.
Bus visibility	With one address space, I/O devices may need to inspect memory references.
Multiple buses	Modern systems often have separate high-speed memory buses, so I/O devices may not see memory addresses directly.

Caching Problem

Device control registers must not be cached like normal memory. If a status register is cached, the CPU may keep reading the old value and never notice that the device became ready.

8 Bus Design Issues



Possible Solutions

On systems with separate memory and I/O buses, memory-mapped I/O needs extra support.

- Try memory first; if memory does not respond, try the I/O buses.
- Add a snooping device that forwards relevant memory-bus addresses to I/O devices.
- Let the memory controller filter address ranges and forward nonmemory addresses to devices.

Final Point

Memory-mapped I/O simplifies software, but it adds hardware and caching trade-offs.

Direct Memory Access

1 Basic Idea

Definition

Direct Memory Access (DMA) is an I/O technique where a DMA controller transfers data between an I/O device and main memory without the CPU copying every byte or word.

Without DMA, the CPU must repeatedly read data from a device controller register and store it in memory. This wastes CPU time, especially for large transfers such as disk blocks.

Main Benefit

DMA lets the CPU start an I/O transfer and then do other work while the DMA controller moves the data.

2 DMA Controller

Role

The **DMA controller** has independent access to the system bus and controls the data transfer after the CPU programs it.

A DMA controller may be built into a device controller, or one shared DMA controller may handle several devices.

DMA register	Purpose
Memory address register	Stores the main-memory address where data should be read or written.
Byte count register	Stores how many bytes remain to transfer.
Control register	Stores direction, device or port, transfer unit, and burst size.

3 Disk Read Without DMA

Step	Action
1	Disk controller reads the disk block into its internal buffer.
2	Disk controller verifies the checksum.
3	Disk controller interrupts the CPU.
4	Operating system copies the block from controller buffer to main memory, one byte or word at a time.

Cost

The CPU is directly involved in the copy loop, so CPU time is spent moving data instead of running useful work.

4 Disk Read With DMA

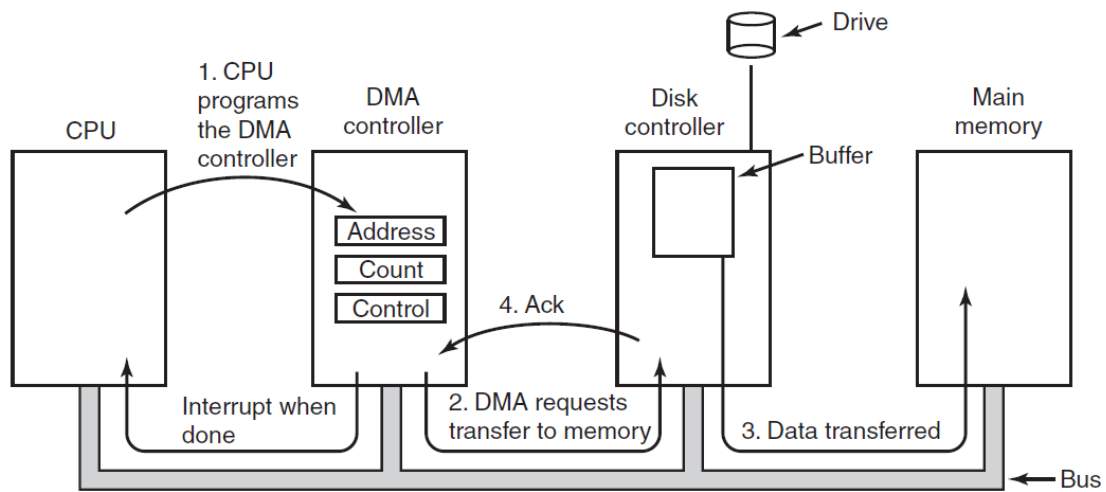


Figure 5-4. Operation of a DMA transfer.

Step	Action
1	CPU loads the DMA registers with address, count, direction, and device information.
2	Disk controller reads the disk block into its internal buffer and checks it.
3	DMA controller requests data from the disk controller.
4	Disk controller writes the data to main memory.
5	Disk controller acknowledges; DMA updates the address and byte count.
6	When the count reaches 0, DMA interrupts the CPU.

Result

When the operating system handles the final interrupt, the block is already in memory.

5 DMA Channels

Multiple Transfers

More advanced DMA controllers can handle multiple transfers using multiple internal register sets called channels.

Each channel stores the parameters for one transfer. Different channels must refer to different device controllers. The DMA controller chooses which pending device to serve next.

Scheduling method	Meaning
Round robin	Serve channels in cyclic order.
Priority based	Serve more important devices first.
Ack lines	Help distinguish acknowledgements from different DMA channels.

6 Transfer Modes

Mode	Description
Cycle stealing	DMA transfers one word at a time and uses occasional bus cycles. The CPU may be delayed slightly.
Burst mode	DMA acquires the bus, transfers many words, and then releases the bus. It is efficient but can block the CPU longer.

Trade-off

Cycle stealing gives better CPU responsiveness. Burst mode gives better transfer efficiency.

7 Fly-By Mode and Alternative Mode

Fly-By Mode

In **fly-by mode**, the DMA controller tells the device controller to transfer data directly to main memory.

Some DMA controllers use a two-step method instead:

1. Device controller sends a word to the DMA controller.
2. DMA controller writes that word to memory.

This needs an extra bus cycle per word, but it is more flexible. It can support device-to-device copies and memory-to-memory copies.

8 Physical and Virtual Addresses

Most DMA controllers use **physical memory addresses**. Therefore, the operating system must translate the target buffer's virtual address into a physical address before programming the DMA controller.

Address Translation

If a DMA controller accepts virtual addresses, it must use the MMU for translation. This is uncommon because the MMU is usually part of the CPU, not the memory system.

9 Why Devices Use Internal Buffers

Disk controllers usually read data into an internal buffer before DMA begins.

Reason	Explanation
Checksum first	The controller can verify the block before transferring it to memory.
Timing control	Disk bits arrive at a fixed rate, even if the system bus is busy.
Simpler design	DMA to memory is not time-critical once the full block is buffered.

Overflow Risk

Without enough buffering, a busy bus can prevent the controller from storing incoming disk data fast enough, causing an overflow error.

10 When DMA May Not Be Used

DMA is not always worth using. On some low-end or embedded systems, removing the DMA controller reduces cost. Also, if the CPU has no other work and is much faster than the DMA controller, CPU-driven I/O may be acceptable.

Final Point

DMA reduces CPU copying overhead, but it adds controller hardware, bus arbitration, address-translation, and buffering concerns.

Interrupts Revisited

1 Basic Idea

Definition

An **interrupt** is a hardware signal that causes the CPU to pause the currently running program and jump to an operating-system routine that handles an event, usually an I/O event.

- Interrupts let devices notify the CPU only when attention is needed.
- Without interrupts, the operating system would repeatedly poll devices.
- Interrupts are important for I/O because devices are much slower than CPU instructions.

Main Purpose

Interrupts allow asynchronous I/O: the CPU can run other work while devices operate, then react when a device signals completion.

2 Interrupt Hardware Structure

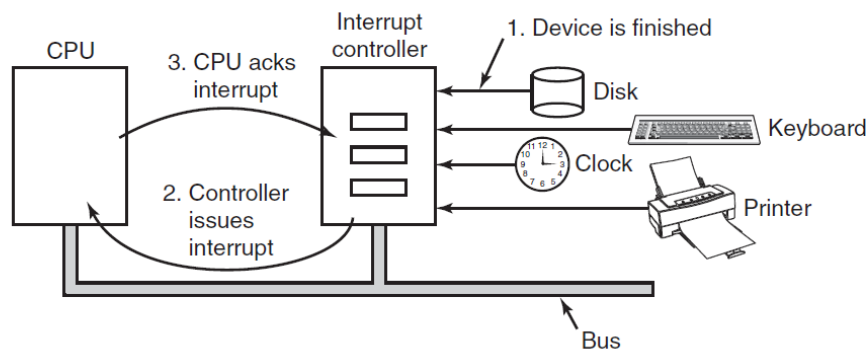


Figure 5-5. How an interrupt happens. The connections between the devices and the controller actually use interrupt lines on the bus rather than dedicated wires.

Component	Role
I/O device	Raises an interrupt request when it finishes work or needs attention.
Interrupt controller	Receives requests, resolves priority, and sends one interrupt signal to the CPU.
CPU	Stops normal execution, saves enough state, and starts the interrupt-service procedure.
Interrupt vector	Table used to find the correct interrupt-service procedure for the requesting device.

3 How an Interrupt Happens

Interrupt Path

The interrupt controller turns a device's request into a CPU-visible interrupt and identifies which service routine should run.

1. A device finishes its assigned work and asserts an interrupt request line.
2. The interrupt controller checks priority and whether another interrupt is active.
3. If the request can be handled, the controller accepts it.
4. The controller places a device-identifying number on the address lines.
5. The controller asserts the CPU interrupt signal.
6. The CPU uses the number as an index into the interrupt vector.
7. The CPU loads the new program counter from the vector entry and starts the interrupt-service procedure.

Interrupt Vector

The **interrupt vector** maps interrupt numbers to handler addresses. Traps and hardware interrupts often share this mechanism after the CPU has selected the vector entry.

4 Priority, Pending Requests, and Acknowledgement

Situation	Controller behavior
No interrupt is active	The controller can immediately signal the CPU.
Another interrupt is being handled	The new request waits until the controller is ready to issue it.
Two devices request together	The request on the higher-priority line is handled first.
Device is not yet serviced	The device continues asserting its interrupt signal.

Important Detail

When a lower-priority device is ignored temporarily, the request should not disappear. The device keeps asserting the interrupt until the CPU eventually services it.

Soon after the interrupt-service procedure starts, it writes a specific value to an I/O port of the interrupt controller. This is called **acknowledging the interrupt**. The acknowledgement tells the controller that it may issue another interrupt when appropriate.

5 State Saved by Hardware

Minimum Saved State

At minimum, the CPU must save the **program counter** (*PC*), because the *PC* tells the CPU where the interrupted program should continue.

Saved item	Why it matters
Program counter	Identifies where execution should resume.
Processor status word	Preserves condition codes, interrupt-enable bits, privilege state, and other CPU status.
General registers	Preserves the interrupted program's working values.
Internal state	May be needed on complex pipelined or superscalar CPUs.

6 Where to Save the State

Place	Advantage	Problem
Internal registers	Fast and avoids immediate memory writes.	A second interrupt may overwrite the saved state unless interrupts stay disabled.
Current stack	Simple because hardware can push state using the current stack pointer.	The stack may be a user stack, invalid, or near a page boundary.
Kernel stack	More likely to be valid, protected, and pinned in memory.	Switching to kernel state may require MMU work and can disturb cache and TLB contents.

Stack Hazard

If hardware saves interrupt state on an invalid user stack, the interrupt itself can cause a fatal error. If the save crosses a page boundary, it may cause a page fault while the CPU is already processing an interrupt.

7 Precise and Imprecise Interrupts

Precise Interrupt

A **precise interrupt** leaves the machine in a well-defined state, so the operating system can restart execution cleanly.

A precise interrupt has four required properties:

1. The *PC* is saved in a known place.
2. All instructions before the instruction pointed to by the *PC* have completed.
3. No instruction beyond the instruction pointed to by the *PC* has finished.

- The execution state of the instruction pointed to by the *PC* is known.

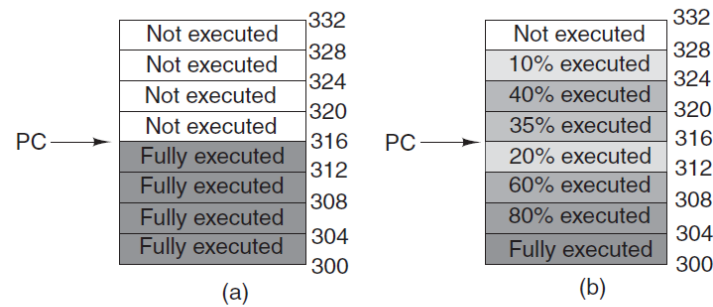


Figure 5-6. (a) A precise interrupt. (b) An imprecise interrupt.

Key Boundary

For a precise interrupt, the saved *PC* marks a clean boundary: instructions before it are complete, and later instructions have no visible completed effects.

8 Why Modern CPUs Make This Hard

- In a pipeline, different instructions occupy different stages at the same time.
- In a superscalar CPU, multiple instructions or micro-operations may execute together.
- Out-of-order execution can let a younger instruction nearly finish while an older one is still waiting.
- To provide precise interrupts, hardware may need retirement, rollback, logging, or shadow register logic.

Operating-System Difficulty

An **imprecise interrupt** does not give a clean restart boundary. The operating system may have to inspect saved internal CPU state to determine what completed and what still needs to be restarted.

9 Design Trade-Off

Type	Meaning and consequence
Precise interrupt	Clear restart boundary at the saved <i>PC</i> . Simpler for the OS, but hardware is more complex.
Imprecise interrupt	No clean boundary around the saved <i>PC</i> . Hardware may be simpler, but OS recovery is harder.

Performance Trade-Off

Precise interrupts move complexity into the CPU. Imprecise interrupts move complexity into the operating system.

Principles of I/O Software

1 Basic Idea

Definition

I/O software is the operating-system layer that hides device details and provides controlled, uniform, and reliable access to input/output devices.

- I/O hardware differs widely from device to device.
- I/O software gives programs a common way to use those devices.
- The OS handles device commands, errors, buffering, sharing, and blocking behavior.
- The goal is convenience for user programs without losing hardware control.

2 Main Goals

Goal	Meaning
Device independence	Programs should access I/O devices without knowing the exact device type.
Uniform naming	Files and devices should be named in a consistent way.
Error handling	Errors should be handled as close to the hardware as possible.
Synchronous interface	User programs should be able to use simple blocking I/O.
Asynchronous support	High-performance programs may need interrupt-driven or non-blocking I/O.
Buffering	The OS may need temporary storage between a device and the final destination.
Device sharing	The OS must handle both shared and dedicated devices correctly.

3 Device Independence

Core Principle

Device independence means a program should work with any suitable I/O device without being rewritten for that device.

A program that reads a file should not care whether the file is stored on a hard disk, DVD, USB stick, or network-backed storage. The program asks for file input; the operating system translates that request into the correct device-specific operation.

Example

The same file-reading program should work whether the input file is stored on a disk, removable drive, or other mounted storage device.

4 Uniform Naming

Definition

Uniform naming means files and devices are identified by simple names, such as strings or integers, instead of by physical device details.

In UNIX-like systems, devices and disks can be integrated into one file-system hierarchy. A USB stick can be mounted at `/usr/ast/backup`; copying to `/usr/ast/backup/monday` then copies data to the USB stick using an ordinary path name.

Command Example

`sort <input >output` should work no matter whether input comes from a disk file or keyboard, and output goes to a disk file or screen.

5 Error Handling

Principle

I/O errors should be handled at the lowest layer that can correctly fix them.

1. The device controller tries to correct hardware-level errors.
2. If the controller cannot fix the problem, the device driver retries or applies recovery.
3. Only if lower layers fail should upper layers be told about the error.

Layer	Error-handling responsibility
Controller	Detect and correct simple hardware errors when possible.
Device driver	Retry operations and apply device-specific recovery.
OS upper layers	Report failures only when low-level recovery fails.
User program	Usually sees only unrecoverable errors.

Transient Errors

Many I/O errors are temporary. A disk read error caused by dust may disappear when the block is read again.

6 Synchronous vs. Asynchronous I/O

Two Views

Physical I/O is often asynchronous, but user programs are easier to write when I/O appears synchronous.

Type	Meaning and use
Synchronous I/O	The calling program blocks until the operation finishes. This is common for ordinary file I/O.
Asynchronous I/O	The CPU starts the operation and continues. Completion is reported later by interrupt or event.

The OS often starts an interrupt-driven physical operation, blocks the user process, runs other work, and later wakes the process when the interrupt reports completion.

7 Buffering

Definition

Buffering means temporarily storing I/O data in memory before it reaches its final source or destination.

- A network packet may need to be stored before the OS knows its final destination.
- Audio output needs buffering so the device does not run out of data.
- Buffering decouples the rate at which data is produced from the rate at which it is consumed.
- Buffering can also reduce performance because it may require extra copying.

Performance Cost

Buffering improves correctness and timing, but extra memory copies can have a major impact on I/O performance.

8 Sharable and Dedicated Devices

Device Sharing

A **sharable device** can safely be used by multiple users or processes at the same time. A **dedicated device** must be assigned to one user or process until the operation is finished.

Device type	Example	OS concern
Sharable	Disk	Many users can have files open on the same disk at the same time.
Dedicated	Printer	Output from different users must not be mixed randomly on the same page.

Final Point

I/O software hides hardware differences while still handling errors, naming, buffering, blocking behavior, and device sharing correctly.

Programmed I/O

1 Basic Idea

Definition

Programmed I/O is an I/O method in which the CPU performs the transfer itself and repeatedly checks the device status until the device is ready.

- It is the simplest form of I/O.
- The CPU does all the active work.
- The OS sends one item of data to the device at a time.
- After each item, the CPU checks whether the device is ready for the next one.
- This repeated checking is called **polling** or **busy waiting**.

Core Point

In programmed I/O, the CPU is tied up until the I/O operation finishes.

2 Printing Example

Scenario

A user process wants to print the eight-character string ABCDEFGH on a printer through a serial interface.

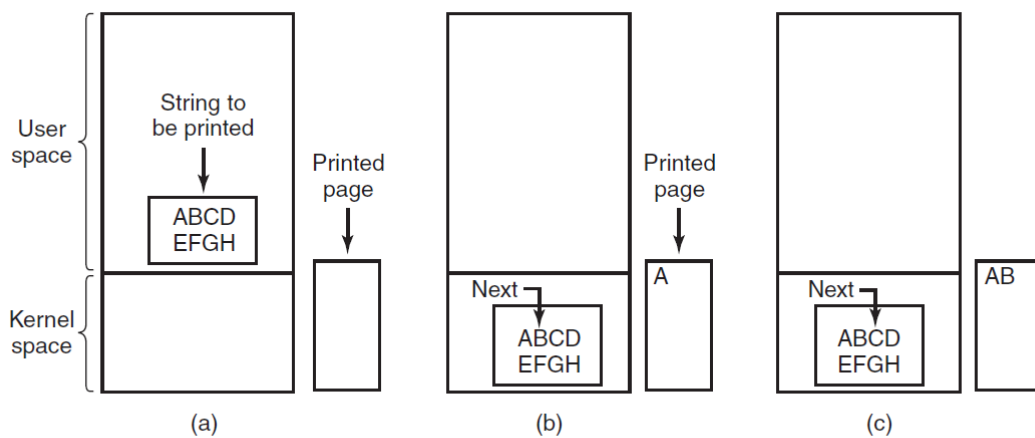


Figure 5-7. Steps in printing a string.

- The user process creates the string in a user-space buffer.

- The process makes a system call to open the printer for writing.
- If the printer is busy, the call may fail or block.
- The OS copies the string from the user buffer to a kernel buffer.
- The kernel buffer is safer for the OS to access while controlling the device.

3 Steps in Programmed I/O

1. Copy data from the user buffer into a kernel buffer.
2. Check whether the printer is available.
3. If the printer is unavailable, wait until it becomes available.
4. Copy the first character to the printer data register.
5. Check the printer status register.
6. If the printer is not ready, keep checking the status register.
7. When the printer becomes ready, copy the next character.
8. Repeat until all characters have been printed.
9. Return control to the user process.

Status Register

The printer status register tells the CPU whether the printer is ready to accept another character.

4 Pseudocode

Writing a String Using Programmed I/O

```
copy_from_user(buffer, p, count);    /* p is the kernel buffer */

for (i = 0; i < count; i++) {
    while (*printer_status_reg != READY)
        ;                          /* wait until printer is ready */

    *printer_data_register = p[i];    /* output one character */
}

return_to_user();
```

5 Polling and Busy Waiting

Polling

Polling means repeatedly reading a device status register to check whether the device is ready.

After sending one character, the CPU checks the printer status register. If the printer is not ready, the CPU checks again and continues this loop until the device becomes ready. During this waiting time, the CPU is not doing useful work for another process.

Main Disadvantage

Programmed I/O can waste CPU time because the CPU waits actively instead of sleeping or running another process.

6 When It Is Acceptable

Case	Why programmed I/O may be acceptable
Very fast device action	If the device only copies data to an internal buffer, waiting time may be very short.
Small transfers	The overhead of interrupts or DMA may be larger than the transfer itself.
Embedded systems	If the CPU has nothing else to do, busy waiting may be simple and acceptable.
Simple hardware	Programmed I/O avoids extra interrupt or DMA complexity.

7 When It Is Inefficient

- It is inefficient when the CPU has other useful work to run.
- It is inefficient for slow devices and long transfers.
- It does not overlap CPU computation with I/O activity.
- It can reduce overall system throughput in multiprogrammed systems.

Better Alternatives

For complex systems, interrupt-driven I/O or DMA is usually better because the CPU does not have to busy-wait for the whole transfer.

8 Quick Comparison

Feature	Programmed I/O
CPU role	CPU directly controls the transfer.
Waiting style	CPU repeatedly polls the device.
Hardware support	Simple; no DMA controller is required.
Main advantage	Easy to understand and implement.
Main disadvantage	Wastes CPU time during busy waiting.
Best use	Small, fast, or embedded-system I/O.

Interrupt-Driven I/O

1 Basic Idea

Definition

Interrupt-driven I/O is an I/O method in which the CPU starts an operation, blocks the requesting process, runs another process, and later resumes I/O work when the device raises an interrupt.

- Programmed I/O wastes CPU time by busy waiting.
- Interrupt-driven I/O avoids most of that waiting.
- The CPU gives the device one item of work and then runs another process.
- When the device is ready again, it interrupts the CPU.

Core Point

Interrupt-driven I/O lets the CPU overlap device waiting time with useful work from other processes.

2 Printer Motivation

Example

Suppose a printer prints 100 characters per second and does not buffer characters.

$$\text{Time per character} = \frac{1 \text{ second}}{100 \text{ characters}} = 0.01 \text{ second} = 10 \text{ msec}$$

After each character is written, the printer needs about 10 msec before accepting the next character. In programmed I/O, the CPU would sit in a polling loop for those 10 msec. That is enough time to perform a context switch and run another process.

3 System Call Path

Initial Print Request

When a process asks to print a string, the OS starts the first character and then blocks the requesting process until the whole string has printed.

1. The user process makes a print system call.
2. The OS copies the user buffer into a kernel buffer.

3. The OS waits until the printer can accept a character.
4. The OS writes the first character to the printer data register.
5. The requesting process is blocked.
6. The scheduler chooses another process to run.

Code During Print System Call

```
copy_from_user(buffer, p, count);
enable_interrupts();

while (*printer_status_reg != READY)
    ;                                /* wait for first character only */

*printer_data_register = p[0];
scheduler();                        /* run another process */
```

4 Interrupt Handler Path

Printer Interrupt

When the printer finishes one character and can accept another, it generates an interrupt.

1. The interrupt stops the currently running process.
2. The CPU saves that process's state.
3. The printer interrupt-service procedure runs.
4. If more characters remain, the handler sends the next character.
5. If no characters remain, the handler unblocks the original user process.
6. The handler acknowledges the interrupt and returns.

Printer Interrupt-Service Procedure

```
if (count == 0) {
    unblock_user();
} else {
    *printer_data_register = p[i];
    count = count - 1;
    i = i + 1;
}

acknowledge_interrupt();
return_from_interrupt();
```

5 What Changes Compared with Programmed I/O

Feature	Programmed I/O	Interrupt-driven I/O
Waiting style	CPU polls until the device is ready.	CPU runs another process while waiting.
Device notification	CPU discovers readiness by checking status.	Device announces readiness by interrupt.
CPU use	CPU can be wasted in a busy loop.	CPU time can be used for other work.
Complexity	Simple.	More complex because interrupts and handlers are needed.

6 Cost and DMA

Main Cost

If an interrupt occurs for every character, the system may spend too much time saving state, running handlers, acknowledging interrupts, and returning.

DMA Idea

Direct Memory Access (DMA) reduces CPU involvement by letting a DMA controller feed data to the device instead of interrupting the CPU for every character.

Feature	Interrupt-driven I/O	DMA
Interrupt frequency	Often one interrupt per character or small unit.	Usually one interrupt per buffer.
CPU involvement	CPU sends each next character from the interrupt handler.	DMA controller performs the repeated transfer.
Main benefit	Avoids busy waiting.	Reduces interrupt overhead and frees the CPU more.
Possible drawback	Too many interrupts can waste CPU time.	DMA may be unnecessary for small transfers.

Final Point

Interrupt-driven I/O improves over programmed I/O by avoiding busy waiting, but DMA improves further by reducing interrupts from many small events to one larger completion event.

I/O Software Layers

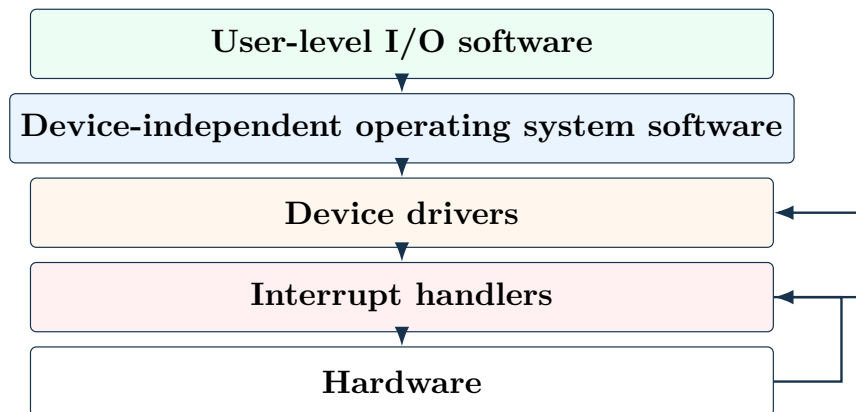
1 Basic Idea

Definition

I/O software layers are the operating-system layers that organize I/O work from user-level requests down to hardware operations and interrupt handling.

- Each layer has a specific function.
- Each layer communicates with adjacent layers through a defined interface.
- The exact design differs between operating systems.
- The main goal is to hide low-level device complexity from most of the OS and from user programs.

2 Layered Structure



Layer	Main responsibility
User-level I/O software	Provides libraries, formatting routines, and user-visible I/O interfaces.
Device-independent OS software	Provides common naming, protection, buffering, allocation, and error handling.
Device drivers	Contains device-specific code and knows controller commands and registers.
Interrupt handlers	Respond to hardware interrupts and wake blocked driver code.
Hardware	Performs the actual data transfer and raises interrupts.

3 Why Use Layers

- Layers separate general I/O policy from device-specific mechanisms.

- User programs do not need to know controller registers or interrupt details.
- Most of the operating system can call uniform I/O interfaces.
- Device-specific complexity is mostly isolated inside drivers.
- Interrupt details are hidden as deeply as possible.

Main Benefit

Layering makes I/O easier to manage because only the lowest layers deal directly with hardware timing, registers, and interrupts.

4 Interrupt Handlers

Role

Interrupt handlers are low-level routines that run when hardware interrupts the CPU.

Interrupts are needed for most real I/O, but they are difficult to handle cleanly. The OS tries to hide interrupts deep inside the kernel. Ideally, higher OS layers see a completed I/O operation, not raw interrupt events.

5 Blocking the Driver

Driver Blocking Model

A driver starts an I/O operation and then blocks until the interrupt reports that the operation has completed.

The driver may block on a semaphore, condition variable, or message. This model works especially well when drivers are kernel processes, because a kernel-process driver has its own state, stack, and program counter.

Blocking method	Matching unblock action
Semaphore	The interrupt handler performs an up operation.
Condition variable	The interrupt handler signals the condition variable.
Message passing	The interrupt handler sends a message to the blocked driver.

Effect of the Interrupt

The interrupt handler performs the low-level interrupt work and then unblocks the driver that was waiting for I/O completion.

6 Interrupt Processing Steps

Not Just IRET

Handling an interrupt is not only acknowledging the interrupt and returning. The OS may need to save state, set up memory context, run a handler, schedule, and restore a process.

1. Save registers and processor status if hardware has not already done so.
2. Set up a context and stack for the interrupt-service procedure.
3. Acknowledge the interrupt controller or reenale interrupts.
4. Copy saved registers into the process table if needed.
5. Run the interrupt-service procedure and read controller status.
6. Choose which process should run next.
7. Set up the MMU and TLB context for that process if needed.
8. Load the selected process's registers and processor status.
9. Start running the selected process.

7 Why Interrupt Processing Is Expensive

- Register state may need to be saved and restored.
- The process table may need to be updated.
- The scheduler may need to choose a new process.
- Virtual memory systems may require MMU and page-table changes.
- TLB entries may need to be changed or reloaded.
- CPU cache behavior may be affected when switching between user and kernel mode.

Final Point

Interrupt handlers are placed at the bottom of the I/O software stack because interrupts are hardware-specific, expensive, and best hidden from higher-level OS code.

Device Drivers

1 Basic Idea

Definition

A **device driver** is device-specific operating-system code that controls an I/O device by issuing commands to its controller and interpreting its status.

- Different devices have different registers, commands, timing rules, and status meanings.
- A mouse driver handles movement and button information.
- A disk driver handles sectors, tracks, cylinders, heads, arm movement, motor timing, and head settling.
- Because devices differ so much, each device type usually needs its own driver.

Core Point

The driver hides device-specific hardware details from the rest of the operating system.

2 Position in the System

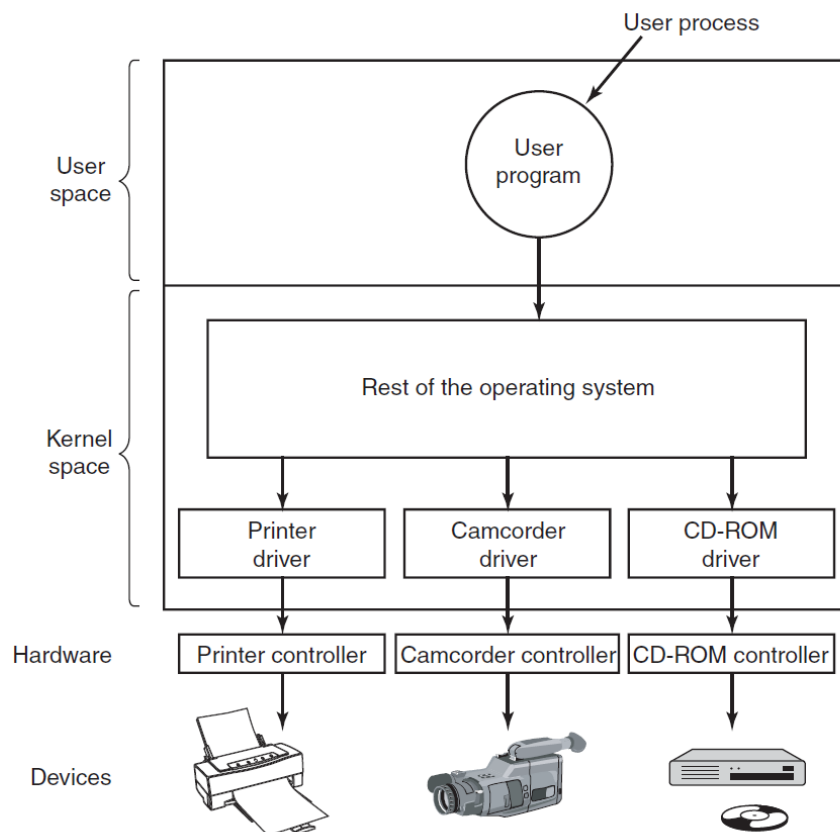


Figure 5-12. Logical positioning of device drivers. In reality all communication between drivers and device controllers goes over the bus.

Device drivers sit below the rest of the operating system. User programs call the operating system, not the driver directly. The rest of the OS sends abstract requests to drivers, and drivers translate those requests into controller commands.

3 One Driver or Many Drivers

Case	Explanation
One device type	A driver normally handles one device type.
Related device class	One SCSI disk driver may handle many SCSI disks of different sizes and speeds.
Unrelated devices	One driver could control unrelated devices, but it is usually a bad design.
Very different devices	A mouse and joystick usually need different drivers.

Design Rule

Putting unrelated devices into one driver makes the driver harder to maintain, test, and reason about.

4 Stacked Drivers and USB

USB Example

USB devices are very different at the top level, but they share lower-level USB communication layers.

Layer	Role in USB-style driver stack
Device-specific API	Provides behavior for mass storage, cameras, keyboards, and other device classes.
Common USB layer	Handles common USB packet and protocol functionality.
USB link layer	Handles low-level signaling and conversion between signals and USB packets.

5 Kernel Drivers vs. User-Space Drivers

Location	Benefit	Cost
Kernel space	Direct access to controller registers and efficient interaction with kernel services.	Buggy drivers can crash or corrupt the kernel.
User space	Better isolation between drivers and the kernel; improves reliability.	Needs system-call support to access registers and may add overhead.

6 Driver Interfaces

Standard Interface

The operating system defines standard procedures that drivers must support so the rest of the OS can call them uniformly.

Driver type	Device examples	Typical operation
Block driver	Disks, SSDs, optical drives	Read or write independently addressable blocks.
Character driver	Keyboards, printers, serial devices	Read or write streams of characters.

7 Main Driver Functions

Function	Meaning
Accept requests	Receive abstract read and write requests from device-independent software.
Validate input	Check that request parameters are legal.
Translate requests	Convert abstract requests into concrete device terms.
Initialize device	Prepare the device before use if needed.
Manage power	Turn devices on, off, or into low-power states.
Issue commands	Write command sequences into controller registers.
Handle completion	Block for interrupts or complete immediately when no wait is needed.
Check errors	Examine controller status and report success or failure.
Start queued work	Select and start another waiting request when possible.

8 Typical Driver Workflow

1. Receive a request from device-independent OS software.
2. Check whether input parameters are valid.
3. Translate the request into device-specific terms.
4. Queue the request if the device is already busy.
5. If idle, examine hardware status and power on the device if required.
6. Issue the required command sequence to controller registers.
7. Wait for completion if the operation takes time.
8. Check for errors, pass data upward, and return status.
9. Start another queued request, or block waiting for the next request.

9 Why Drivers Are Complicated

Reentrant Drivers

A driver may be interrupted while it is already running, and the interrupt may cause that same driver to run again. Shared driver data structures must be protected carefully.

Hot-pluggable devices add another problem: a device may be removed while the driver is performing I/O. The driver must abort the current transfer, cancel pending requests, and tell callers that the device is gone.

Kernel Interaction

Drivers normally cannot make ordinary system calls, but they can call selected kernel procedures for pinned memory, timers, DMA, interrupt control, logging, and error reporting.

Final Point

Device drivers are the boundary between general OS I/O code and device-specific hardware behavior.

Device-Independent I/O Software

1 Basic Idea

Definition

Device-independent I/O software is the part of the operating system that performs I/O functions common to many devices and presents a uniform interface to upper layers.

- Some I/O code must be device-specific and belongs in device drivers.
- Other I/O work is common across devices and belongs in device-independent OS software.
- The exact boundary between drivers and device-independent software depends on the system.
- The main purpose is to make different devices look similar to the rest of the OS.

2 Main Functions

Function	Meaning
Uniform interface	Makes drivers of the same class support the same OS-visible operations.
Buffering	Temporarily stores input or output data to handle timing, paging, and rate differences.
Error reporting	Provides a common framework for reporting and escalating I/O errors.
Dedicated devices	Controls exclusive-use devices such as printers.
Logical block size	Hides physical sector-size differences from higher layers.

Core Point

Device-independent I/O software hides differences that higher layers should not need to know.

3 Uniform Driver Interfaces

Problem

If every driver has a different interface, the operating system must be modified whenever a new device or driver is added.

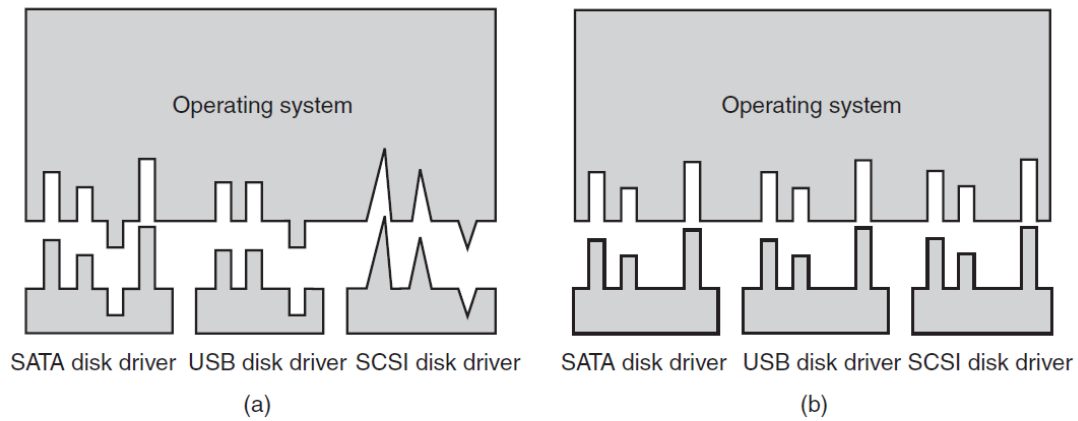


Figure 5-14. (a) Without a standard driver interface. (b) With a standard driver interface.

With a standard interface, all drivers in a class obey the same contract. Driver writers know what procedures they must provide, and the OS can call drivers uniformly through known entry points.

Driver Interface Table

For each device class, the OS can define a table of function pointers that every driver in that class must provide.

Device class	Typical standard operations
Disk class	Read block, write block, format, power control, status check.
Printer class	Write character stream, check status, start output, stop output.
Network class	Send packet, receive packet, set address, report link status.

4 Device Naming and Protection

Uniform Naming

Device-independent software maps symbolic device names to the correct driver and device unit.

In UNIX-like systems, devices appear as special files in the file system. A name such as `/dev/disk0` identifies a special file whose i-node contains device-identifying information.

Major and Minor Numbers

Major number = which driver. Minor number = which device controlled by that driver.

Device access must also be protected. If devices are named objects, normal file-protection rules can apply, and the system administrator can set permissions for each device.

5 Buffering

Definition

Buffering means using temporary memory storage to hold I/O data before it reaches its final source or destination.

- Buffering is needed for both block devices and character devices.
- It handles mismatches between device speed and process speed.
- It can avoid waking a user process for every small piece of data.
- It can solve problems caused by user pages being paged out.
- It can improve output behavior by letting the user reuse a buffer quickly.

6 Input Buffering Methods

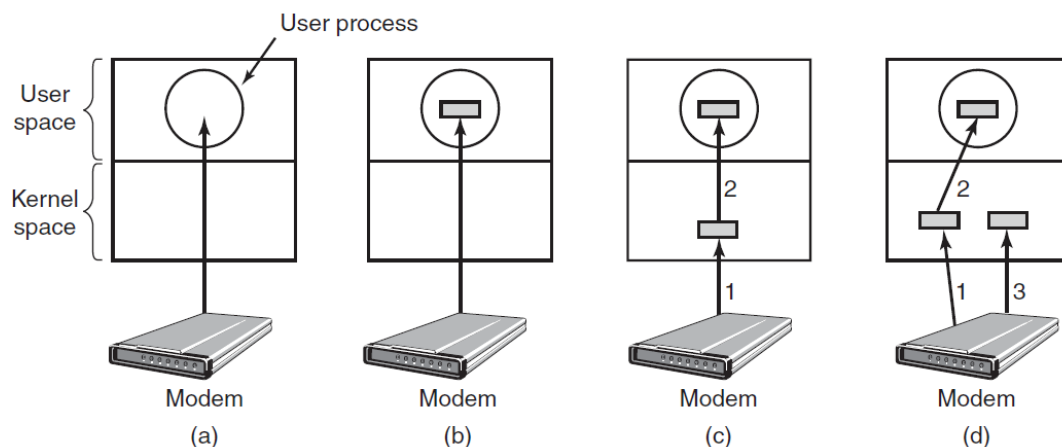


Figure 5-15. (a) Unbuffered input. (b) Buffering in user space. (c) Buffering in the kernel followed by copying to user space. (d) Double buffering in the kernel.

Method	Meaning and drawback
Unbuffered input	The user process waits for one character at a time and may be restarted for every character.
User-space buffer	Reduces wakeups, but the buffer page may be paged out when input arrives.
Kernel buffer	The interrupt handler stores characters in a kernel buffer; data are copied to user space later.
Double buffering	Two kernel buffers alternate, so one can be copied while the other receives new input.

Double Buffering

Double buffering avoids losing input while one full kernel buffer is being copied to user space.

7 Circular Buffers and Output Buffering

Circular Buffer

A **circular buffer** is a fixed memory region with two moving pointers: one for inserting new data and one for removing old data.

Circular buffers are common in continuous data streams such as networking. Hardware or the driver advances the input pointer as data arrive, while the OS advances the output pointer as data are consumed. Both pointers wrap around at the end.

For output, the OS can copy a user's data into a kernel buffer and unblock the process immediately. The actual device output continues later from the kernel buffer.

Buffer Ownership

Copying output to a kernel buffer makes it safe for the user process to reuse its original buffer immediately.

8 Cost of Too Much Buffering

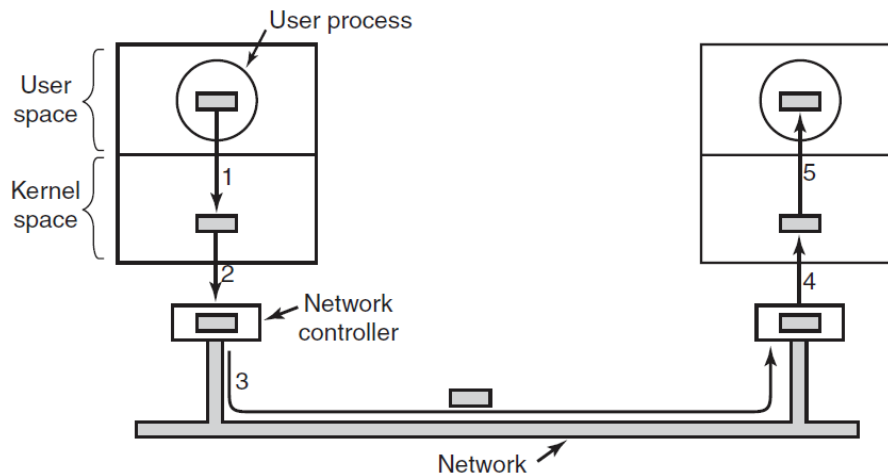


Figure 5-16. Networking may involve many copies of a packet.

1. Sender copies the packet from user space to a kernel buffer.
2. The driver copies the packet to the network controller.
3. The network controller sends the packet across the network.
4. The receiver copies the packet from its controller to a kernel buffer.
5. The receiver copies the packet from the kernel buffer to the receiving process's buffer.

Performance Cost

Buffering is useful, but repeated copying can slow down I/O because the copies happen sequentially.

9 Error Reporting

Goal

Device-independent software provides a common error-reporting framework, even though many I/O errors are handled inside specific drivers.

Error type	Handling approach
Programming error	Return an error code to the caller.
Invalid operation	Examples include writing to an input device or reading from an output device.
Invalid parameter	Examples include an invalid buffer address or invalid device number.
Actual I/O error	The driver tries recovery first; if it cannot recover, the problem is passed upward.
Critical system damage	The system may have to print an error message and terminate.

10 Dedicated Device Allocation and Block Size

Dedicated Device

A **dedicated device** can be used by only one process at a time.

Printers are a common example. The OS may fail an open request if the device is unavailable, or it may block the caller in a queue until the device becomes available.

Logical Block Size

Device-independent software can present a uniform logical block size even when physical devices use different sector sizes.

The OS can treat several physical sectors as one logical block. Higher layers then work with abstract devices that share one block size, even when the physical hardware differs.

Final Point

Device-independent I/O software standardizes interfaces, names, protection, buffering, error handling, dedicated-device allocation, and logical block sizes across different devices.

User-Space I/O Software

1 Basic Idea

Definition

User-space I/O software is the part of the I/O system that runs outside the kernel, mainly as library procedures linked with user programs or as user-level service programs.

- Most I/O software is inside the operating-system kernel.
- A smaller part runs in user space.
- User-space I/O includes library routines such as `write`, `printf`, and `scanf`.
- It can also include user-level programs such as printer daemons and network daemons.
- User-space I/O sits at the top of the I/O software stack.

Core Point

User-space I/O software prepares, formats, and queues I/O requests before the kernel and device layers perform the actual hardware work.

2 Library Procedures for System Calls

System Call Wrapper

Many I/O system calls are invoked through library procedures that package parameters and enter the kernel.

```
count = write(fd, buffer, nbytes);
```

The procedure `write` may be linked into the program binary, or it may be loaded dynamically during execution. The wrapper places parameters where the system-call mechanism expects them and then executes the system-call entry instruction.

3 Formatting I/O

Formatting Work

Some user-space I/O procedures do real work before calling the kernel, especially input and output formatting.

- `printf` takes a format string and values.
- It builds an ASCII character string.

- It then calls `write` to output the resulting string.
- `scanf` reads input and stores parsed values into variables described by a format string.

Why Formatting Is User-Level

Formatting does not require direct hardware access, so it can safely be done by user-space library code.

4 Printf Example

Example Statement

```
printf("The square of %3d is %6d\n", i, i*i);
```

- The literal text `The square of` is copied into the output string.
- The value of `i` is formatted as a 3-character decimal field.
- The literal text `is` is added.
- The value `i2` is formatted as a 6-character decimal field.
- A newline character is appended.
- The completed string is passed to `write`.

Important Detail

`printf` does not print directly to the hardware. It formats data, then uses lower I/O layers through a system call.

5 Spooling

Definition

Spooling is a user-level technique for managing dedicated I/O devices by placing work in a queue directory and letting a daemon process perform the actual device access.

Spooling is useful for dedicated devices such as printers. Direct printer access is risky because a process could open the printer and keep it unavailable for hours. Instead, users place complete print jobs in a spooling directory, and a daemon is the only process allowed to open the printer's special file.

6 Printer Spooling Flow

1. User process generates the complete file to print.
2. User process places the file in the spooling directory.

3. The printer daemon notices or selects the queued file.
4. The daemon opens the printer special file.
5. The daemon sends the file to the printer.
6. When done, the daemon moves to the next queued job.

Spooling Benefit

Spooling turns a dedicated device into a queued service, so many users can submit work without directly owning the device.

7 Other Uses of Spooling

Spooling is not only for printers. Network file transfer can use a network daemon and a network spooling directory. News systems historically used spooling to queue messages for later distribution. These systems can run outside the operating-system kernel.

8 Complete I/O Software Stack

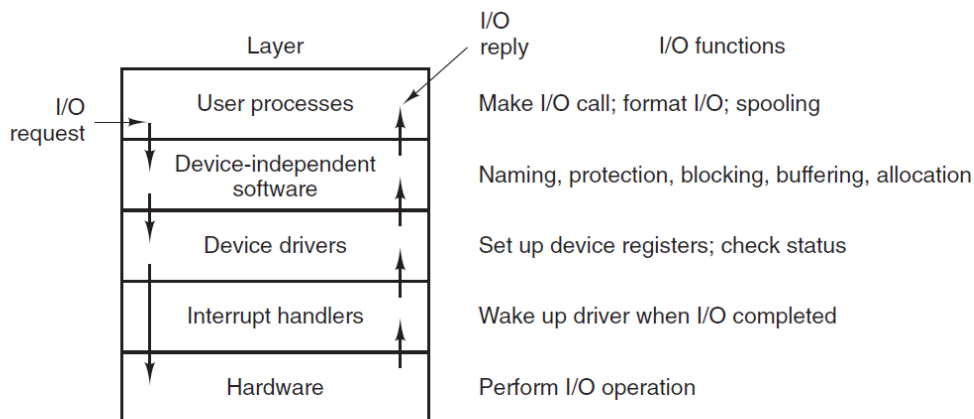


Figure 5-17. Layers of the I/O system and the main functions of each layer.

9 Control Flow for a Read

1. A user program requests a block from a file.
2. The kernel is entered through an I/O system call.
3. Device-independent software checks whether the block is already in the buffer cache.
4. If the block is not cached, the device driver is called.
5. The driver issues the hardware request to the disk controller.
6. The requesting process blocks.

7. When the disk finishes, hardware raises an interrupt.
8. The interrupt handler identifies the device and extracts status.
9. The sleeping process or driver is awakened.
10. The I/O request is finished and the user process continues.

10 Layer Responsibilities

Layer	Responsibility
User processes	Make I/O calls, perform formatted I/O, and use spooling services.
Device-independent OS software	Handles naming, protection, blocking, buffering, and allocation.
Device drivers	Set up controller registers and check device status.
Interrupt handlers	Wake up drivers or processes when I/O completes.
Hardware	Performs the actual I/O operation.

Final Point

User-space I/O software is the top layer of the I/O system: it formats requests, invokes system calls, and can manage queued I/O services such as spooling.